Ch. 6 Notes
Trees and Binary Search Trees

Vocabulary

- **Root** – no parent.
- **Leaf** – no children.
- **Depth** of a node: edges from root. Root depth = 0.
- **Height** of a tree: max depth. Empty tree $= -1$.
- **Level** k: all nodes at depth k.
- **Full**: every node has 0 or 2 children.
- **Complete**: all levels full except possibly the last, which fills left-to-right.
- **Perfect**: all levels full.

Perfect binary tree of height h: $2^{h+1} - 1$ nodes; 2^h leaves.

Node type

```
struct TreeNode {
    int key;
    TreeNode* left  = nullptr;
    TreeNode* right = nullptr;
    // optional: TreeNode* parent;
};
```

BST invariant

For every node n:

- every key in the left subtree is $< n.key$;
- every key in the right subtree is $> n.key$.

(Duplicates disallowed by convention; if allowed, pick one side and document it.)

BST search

```
TreeNode* bstSearch(TreeNode* root, int
    k) {
    while (root && root->key != k)
        root = (k < root->key) ? root->
            left
                                : root->
                                    right;
    return root;
}
```

Cost: $O(h)$ – $O(\log n)$ balanced, $O(n)$ degenerate.

BST insert

Walk as if searching; when you fall off a null pointer, that's where the new node goes.

```
void bstInsert(TreeNode*& root, int k) {
    if (!root) { root = new TreeNode{k};
         return; }
    if (k < root->key)  bstInsert(root->
        left,  k);
    else if (k > root->key) bstInsert(
        root->right, k);
    // else duplicate: policy decision
}
```

BST remove: four cases

1. **Leaf.** Delete the node; parent's pointer → null.
2. **Only left child.** Parent's pointer → that child.
3. **Only right child.** Parent's pointer → that child.
4. **Two children.** Find *in-order successor* (leftmost of right subtree). Copy its key into the node to be deleted. Remove the successor (it has at most one child → case 1 or 3).

Why successor? It's guaranteed > everything in left subtree and < everything in right subtree – preserves the BST invariant.

Traversals

Order	Visit pattern	Use
Pre-order	root, L, R	copy/serialize
In-order	L, root, R	sorted output
Post-order	L, R, root	delete tree

All three are $O(n)$; recursion depth $O(h)$.

In-order, iterative (useful!)

```
void inorder(TreeNode* root) {
    std::stack<TreeNode*> s;
    TreeNode* cur = root;
    while (cur || !s.empty()) {
        while (cur) {
            s.push(cur); cur = cur->left
                ;
        }
        cur = s.top(); s.pop();
        visit(cur);
        cur = cur->right;
    }
}
```

Set-ADT order ops on a BST ($O(h)$ each)

`find_min` – walk left until null.
`find_max` – walk right until null.
`find_next(x)` (in-order successor):

- if `x->right` non-null: leftmost of right subtree;
- else: walk up via parent until you come from the left.

`find_prev` – symmetric. *This is what hash tables can't do* – they cost $O(n)$ for any of these.

Height vs. balance

Best: $h = \lfloor \log_2 n \rfloor \to O(\log n)$ ops.
Worst: ascending/descending insert sequence → "rope" tree, $h = n - 1$, ops become $O(n)$.
Random insertion order: expected $h = O(\log n)$ (CLRS Thm 12.4) – a vanilla BST built from random data already behaves like a balanced one on average. *Production uses red-black (ch. 9) anyway for the worst-case guarantee.*
Balance = the difference. Balance factor of a node = height(left) − height(right). AVL enforces $|bf| \leq 1$.

BST vs. hash table vs. sorted array

	BST	hash	sorted arr
find	$O(\log n)$	$O(1)^*$	$O(\log n)$
insert	$O(\log n)$	$O(1)^*$	$O(n)$
sorted iter	$O(n)$	$O(n \log n)$	$O(n)$
range query	yes	no	yes
min/max	$O(\log n)$	$O(n)$	$O(1)$

* expected. BST numbers assume balance.

Parent pointers: why/why not

With parent: makes remove cleaner, supports iterator-style traversal, required for AVL / red-black rotations. Costs one extra pointer per node.
Without parent: smaller nodes; reach prev/next via recursion stack or reference-to-pointer idiom.

Top gotchas

- Quoting BST ops as $O(\log n)$ without saying *balanced*.
- Removing with two children by replacing with the *root* of a subtree (invalid) instead of the in-order successor.
- Forgetting that insertion order determines tree shape; sorted input → linear chain.
- Using `TreeNode*` (value) instead of `TreeNode*&` (reference) when a function needs to alter the parent's pointer.
- Memory leaks from not deleting the tree (use post-order).